

Submission Worksheet

Submission Data

Course: IT202-008-S2026

Assignment: IT202 Milestone 1 - 2026

Student: Stephane E. (se283)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/30/2026 9:46:36 PM

Updated: 5/4/2026 8:53:25 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone1-2026/se283>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-1-2026/view/se283>

Instructions

- Overview Link: <https://youtu.be/IDtqdaLwcVg>

1. Refer to Milestone1 of this doc:

<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWWwfo/view>

2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
 1. `git checkout Milestone1` (ensure proper starting branch)
 2. `git pull origin Milestone1` (ensure history is up to date)
5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone1`
 4. On Github merge the pull request from Milestone1 to qa
 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout qa`
11. `git pull origin qa`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

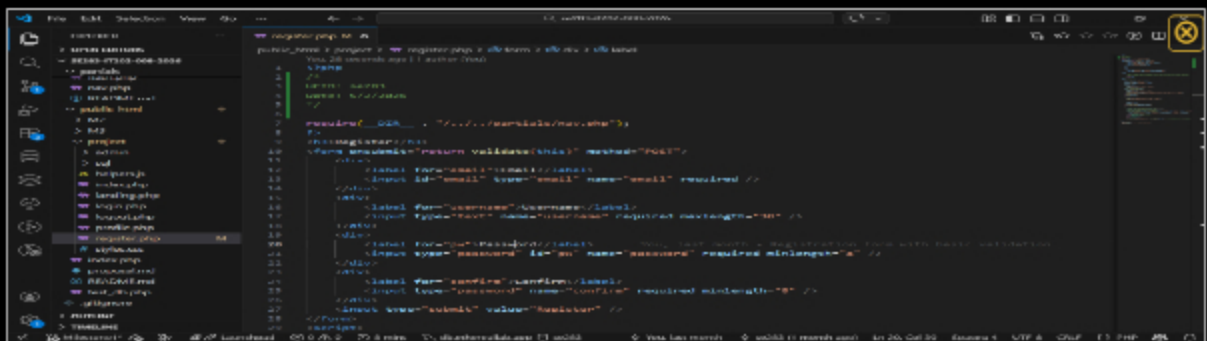
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

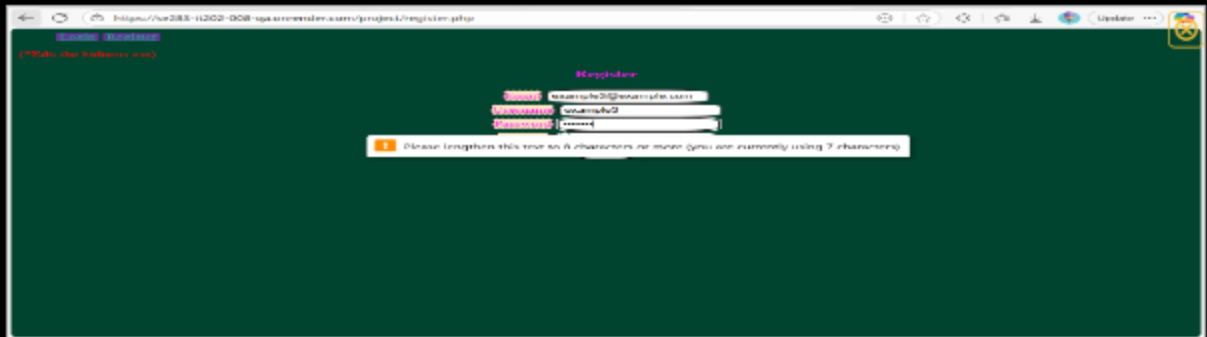


```
register.php:14
<!-- HTML validation -->
<div class="form" style="border: 1px solid #ccc; padding: 10px; margin: 10px auto; width: 400px;">
    <div style="text-align: center; margin-bottom: 10px;">
        <h3 style="margin: 0;">Register
    </div>
    <div>
        <div style="margin-bottom: 10px;">
            <input type="text" value="Username"/>
        </div>
        <div style="margin-bottom: 10px;">
            <input type="email" value="Email"/>
        </div>
        <div style="margin-bottom: 10px;">
            <input type="password" value="Password"/>
        </div>
        <div style="margin-bottom: 10px;">
            <input type="password" value="Confirm Password"/>
        </div>
        <div style="text-align: center;">
            <input type="submit" value="Register"/>
        </div>
    </div>
    <div style="margin-top: 10px;">
        <div style="margin-bottom: 5px;">
            <div style="border: 1px solid #ccc; padding: 2px; display: inline-block;">
                <div style="background-color: #f00; color: #fff; padding: 2px 5px; font-weight: bold;">Error
```

Show the code related to the register page's form (HTML validation)



Show examples of each validation message [email format]



Show examples of each validation message [password format]



Saved: 5/4/2026 4:19:49 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I used standard HTML attributes like `type="email"`, `required`, and `minlength="8"` inside the input tags. This solves the requirement by giving the user instant feedback directly in their browser. It prevents the form from being sent to the server if a field is empty, the email is missing an "@" sign, or the password is too short, ensuring only basic, correctly formatted data is submitted.



Saved: 5/4/2026 4:19:49 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

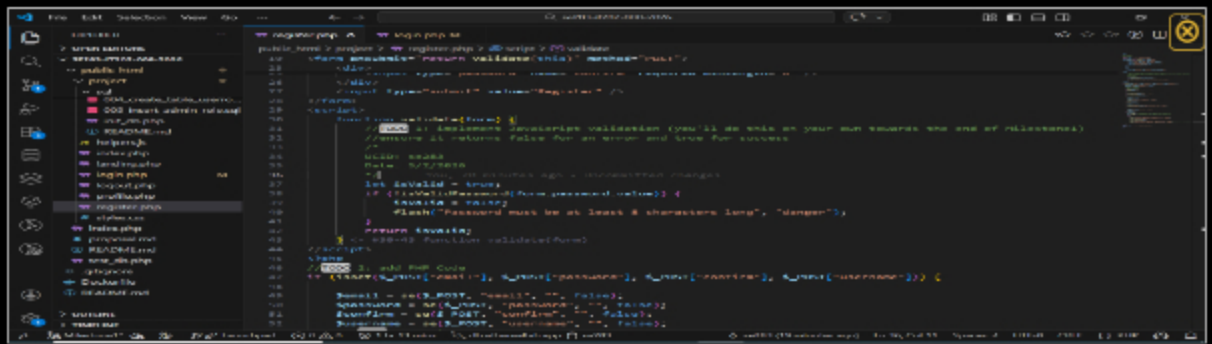
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's `validate()` function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



```
function validate() {  
    // Check if fields are empty  
    if (empty($_POST['username']) || empty($_POST['password']) || empty($_POST['confirm'])) {  
        return false;  
    }  
    // Check if password is at least 8 characters long  
    if (strlen($_POST['password']) < 8) {  
        return false;  
    }  
    // Check if password and confirm match  
    if ($_POST['password'] != $_POST['confirm']) {  
        return false;  
    }  
    // If all checks pass, return true  
    return true;  
}
```

Show the code related to the register page's validate() function

 Saved: 5/4/2026 8:50:27 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I added a JavaScript function called validate(form) that runs the moment the user clicks the "Register" button. This solves the requirement by allowing a final check on the user's computer before the page even reloads. For example, it can quickly check if the "Password" and "Confirm" fields match and alert the user if they don't, which saves time and prevents the server from doing unnecessary work.

 Saved: 5/4/2026 8:50:27 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

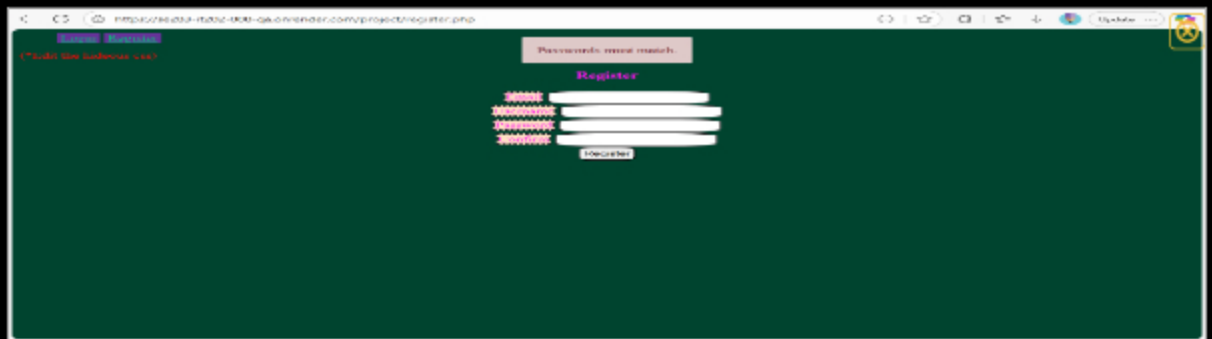
Details:

- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form

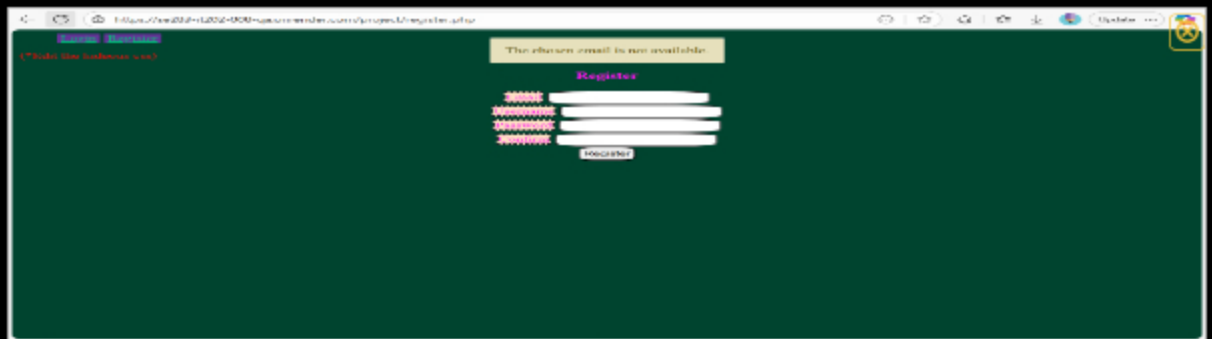
- tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use

```
1 // validate email format
2 function validateEmail($email) {
3     return filter_var($email, FILTER_VALIDATE_EMAIL);
4 }
5
6 // validate password strength
7 function validatePassword($password) {
8     return preg_match('/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@_!#$%^&*()~`{|}~']).{8,}$/', $password);
9 }
10
11 // validate username
12 function validateUsername($username) {
13     return preg_match('/^[a-zA-Z0-9_]{3,15}$/', $username);
14 }
15
16 // validate email not in use
17 function emailNotInUse($email) {
18     $stmt = $db->prepare('SELECT * FROM users WHERE email = ?');
19     $stmt->execute($email);
20     return $stmt->rowCount() == 0;
21 }
22
23 // validate username not in use
24 function usernameNotInUse($username) {
25     $stmt = $db->prepare('SELECT * FROM users WHERE username = ?');
26     $stmt->execute($username);
27     return $stmt->rowCount() == 0;
28 }
29
30 // register user
31 function register($username, $email, $password) {
32     if (!validateEmail($email) || !validatePassword($password) || !validateUsername($username)) {
33         return false;
34     }
35     if (!emailNotInUse($email) || !usernameNotInUse($username)) {
36         return false;
37     }
38     $password = password_hash($password, PASSWORD_BCRYPT);
39     $stmt = $db->prepare('INSERT INTO users (username, email, password) VALUES (?, ?, ?)');
40     $stmt->execute($username, $email, $password);
41     return true;
42 }
```

Show the code related to the register page's PHP validation



Show examples of each validation message [password doesn't match confirm]



Show examples of each validation message [email taken]



Show examples of each validation message [username taken]



Show examples of each validation message [password format]



Saved: 5/4/2026 5:10:36 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I used PHP code (like `isset()` and `is_valid_email()`) to check the data once it reaches the server. This solves the requirement of security because it acts as a gatekeeper that cannot be bypassed. It manually inspects the data to ensure it is safe, hashes the password for protection, and checks the database to make sure the email or username isn't already taken before officially creating the account.



Saved: 5/4/2026 5:10:36 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include the render.com prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

<https://github.com/se283/se283->



URL

<https://github.com/se283/se283->



<https://github.com/se283/se283->

URL #2

<https://se283-it202-008->



URL

<https://se283-it202-008->



[prod.onrender.com/project/register.php](https://se283-it202-008-prod.onrender.com/project/register.php)

URL #3

<https://github.com/se283/se283-it202-008-202168>



<https://github.com/se283/se283-it202-008-202168>



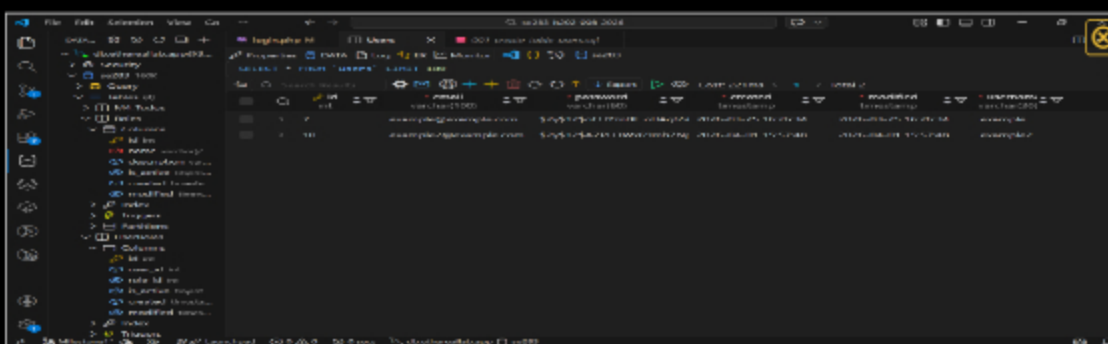
Saved: 5/4/2026 12:41:17 PM

Task #3 (0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)



Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capt



Saved: 5/4/2026 1:16:03 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

Show the code related to the login page's form (HTML validation)

terminal

```

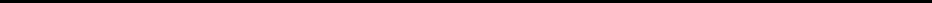
root@kali:~#

```

Shower brightness: Your laptop lid is closed. You are currently using a Chromebook.

Show examples of each validation message [password format]

Show examples of each validation message [no email input]



The screenshot shows a web browser window with the address bar displaying the URL: <https://a0283.0300.008.sgaarnment.com/projects/login.php>. The browser's address bar also shows the GitHub logo and the text "GitHub". The browser's address bar also shows the text "GitHub".



Show examples of each validation message [no password input].PNG



Saved: 5/4/2026 8:24:10 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I used HTML attributes like `type="text"` for the email/username field and `required` with `minlength="8"` for the password. This solves the requirement by checking the data before it even leaves the user's computer. It ensures the user doesn't accidentally leave a box empty or enter a password that is way too short, which helps them fix mistakes immediately.



Saved: 5/4/2026 8:24:10 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

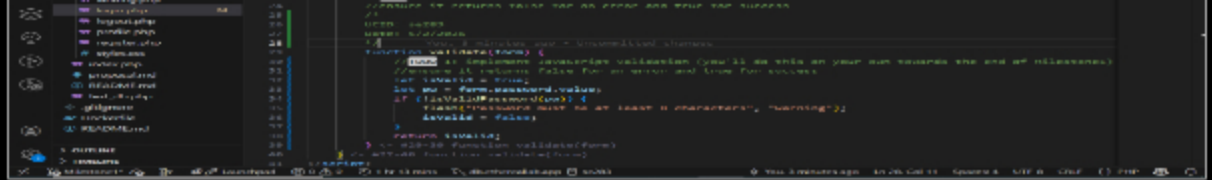
Part 1:

Progress: 100%

Details:

- Show the code related to the login page's `validate()` function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag





Show the code related to the login page's validate() function



Saved: 5/4/2026 8:53:25 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I included a JavaScript function called validate(form) that starts as soon as the user clicks the "Login" button. This satisfies the requirement by allowing the website to do one last quick check on the browser side. While it currently returns true, it is set up so I can add rules, like checking for special characters, to stop the form from sending if the data doesn't look right, saving the server from doing extra work.



Saved: 5/4/2026 8:53:25 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

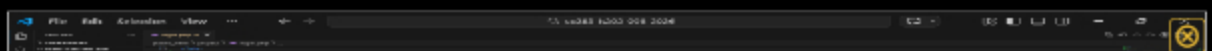
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch (the one that checks against the DB hash)

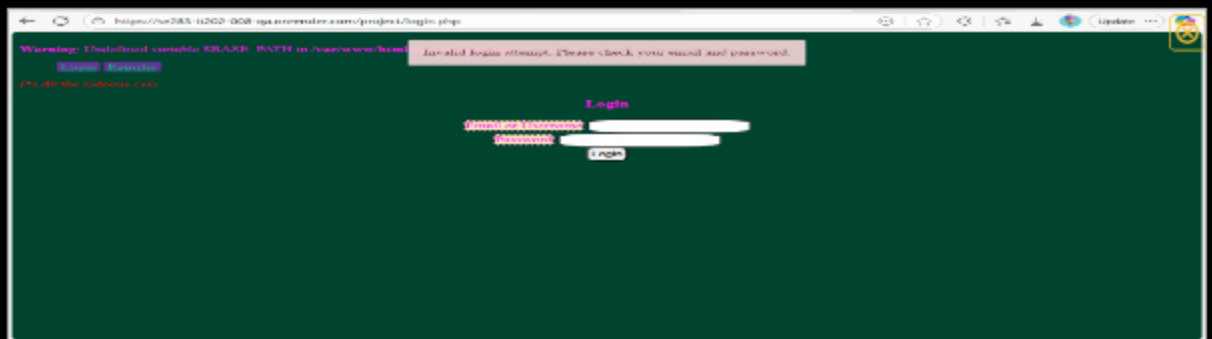


```

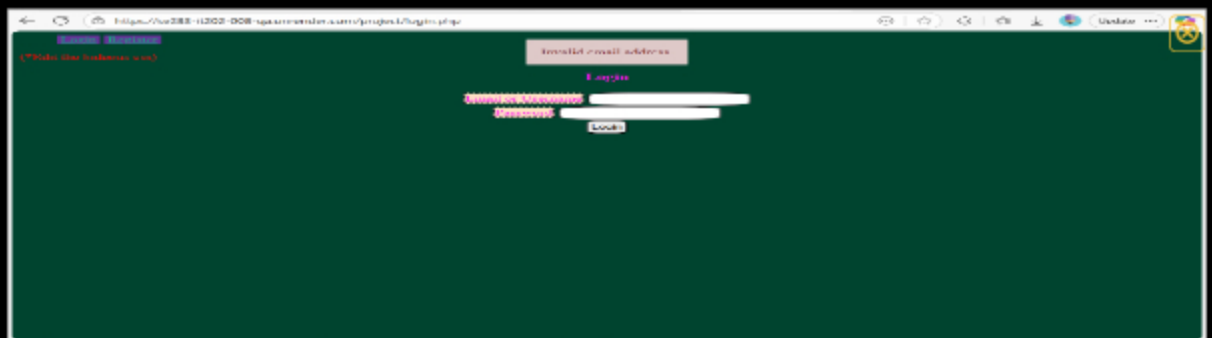
1 <?php
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...

```

Show the code related to the login page's PHP validation



Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to



Show examples of each validation message [invalid email]

 Saved: 5/4/2026 5:46:53 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I used PHP code to check the data once it reaches the server. This solves the requirement by acting as a strict guard that verifies the email and password against the database. It uses password_verify to check the secret hashed password and ensures the account exists before letting the user in. This protects the site even if someone tries to bypass the browser's rules.

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

<https://github.com/se283/se283->



URL

<https://github.com/se283/se283->



<https://github.com/se283/se283-> it202-008-2026 Milestone1/public_html/project/login.php

URL #2

<https://se283-it202-008-prod.onrender.com/project/login.php>



URL

<https://se283-it202-008-prod.onrender.com/project/login.php>



URL #3

<https://github.com/se283/se283-> it202-008-2026



URL

<https://github.com/se283/se283->



Saved: 5/4/2026 12:43:04 PM

Task #3 (0.67 pts.) - User Session Evidence

Progress: 100%

Details:

- From the render.com logs (Logs tab), capture the session error_log
- It should show the id, username, email, and roles



From the render.com logs (Logs tab), capture the session error_log() output. It should show the id, username, email, and roles.



Saved: 5/4/2026 4:36:00 PM

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

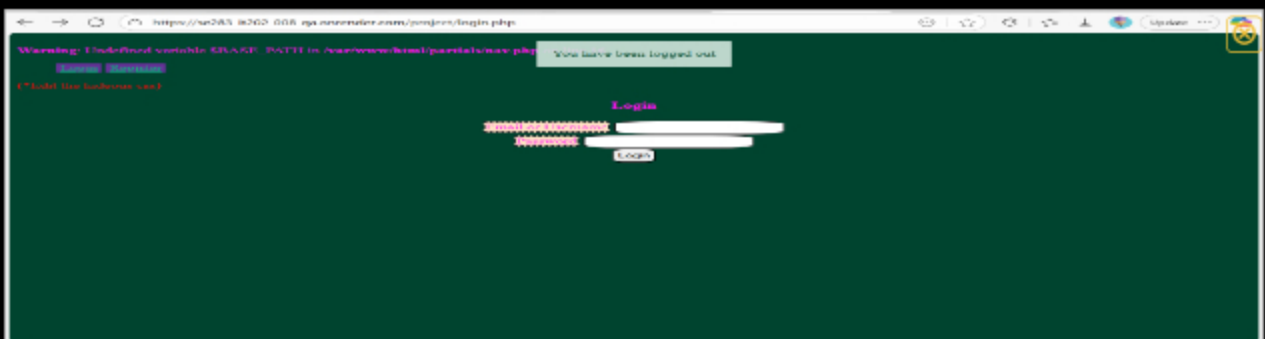
Progress: 100%

Part 1:

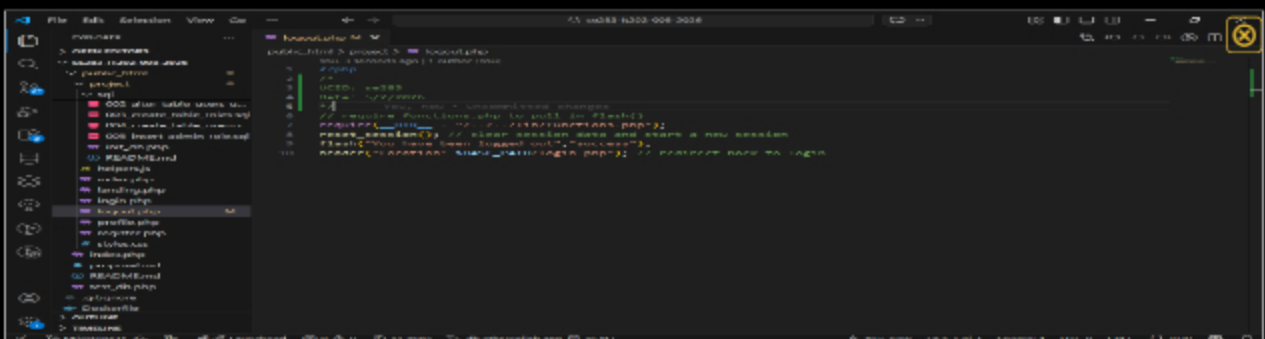
Progress: 100%

Details:

- Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)
- Show the code snippet of the logout page



Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)



Show the code snippet of the logout page



Saved: 5/4/2026 1:52:00 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/se283/se283-it202-008/2026/>



URL

<https://github.com/se283/se283-i>



Saved: 5/4/2026 1:52:00 PM

⇒ Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

The logout process works by "clearing the memory" of the website so it no longer knows who you are. When you click the logout link, the logout.php file runs a special command called `reset_session()`. This command wipes out all the saved info about your user account—like your username and your special permissions—from the server's temporary storage. Because that information is gone, the user is officially logged off and the website treats them like a total stranger again. Finally, the site sends the user back to the login page and shows a simple message saying they have signed out successfully.



Saved: 5/4/2026 1:52:00 PM

Section #4: (2 pts.) Feature: Security Rules

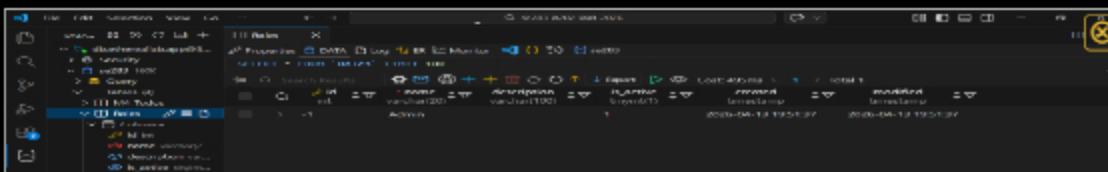
Progress: 100%

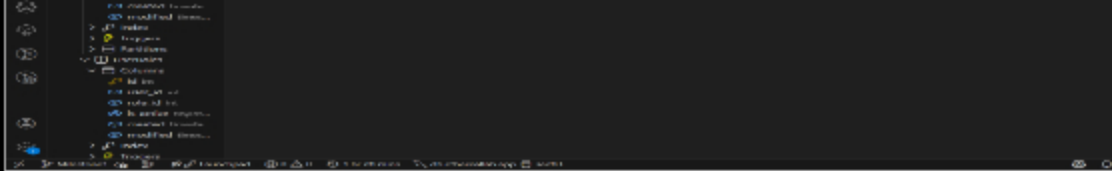
📁 Task #1 (1 pt.) - Database Tables

Progress: 100%

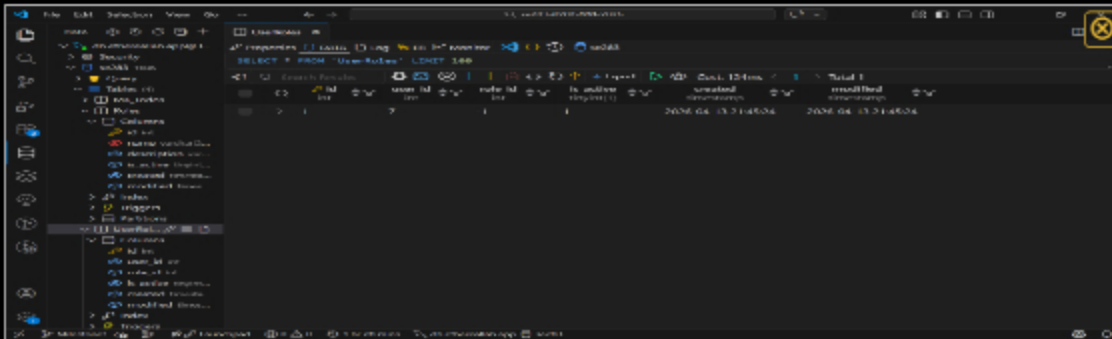
Details:

- From the vs code mysql tool, show both the `Roles` and `UserRoles`





From the vs code mysql tool, show the 'Roles' table with data in them



From the vs code mysql tool, show the 'UserRoles' table with data in them

Saved: 5/3/2026 11:17:54 PM

Task #2 (1 pt.) - Demo Security Rules

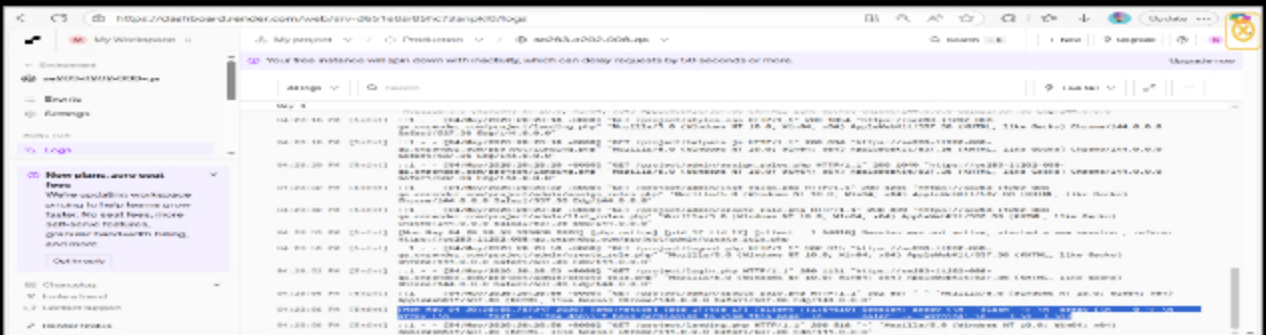
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the message related to needing to be logged in (i.e., try to manually
- Show the message related to not having the proper permission/role (i.e.,
- Include a snippet of the login check function
- Include a snippet of the role check function

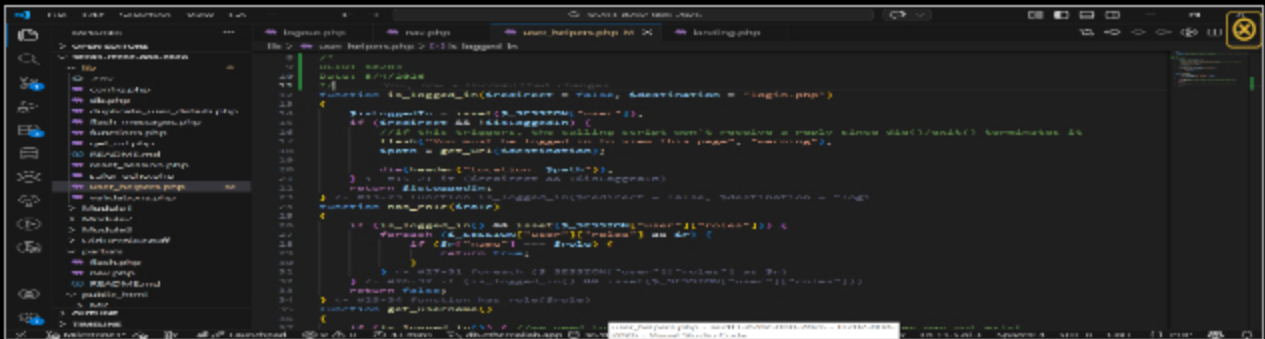


Show the message related to needing to be logged in (i.e., try to manually access a login protected page while logged out)





Show the message related to not having the proper permission/role (i.e., try to manually access a role protected page while not



Include a snippet of the login check function and the role check function



Saved: 5/4/2026 8:02:09 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/se283/se283-it202-008-2026/>



URL

<https://github.com/se283/se283-i>



Saved: 5/4/2026 8:02:09 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

The login check works by looking at the website's "memory" (called a session) to see if a user is currently active. The `is_logged_in()` function checks if the user key exists in that memory; if it's

missing, the code can automatically send the person back to the login page so they can't see private data. The role check goes one step further using the `has_role()` function. It looks inside that same memory to see if the logged-in user has a specific "tag" or permission, like "Admin." It loops through a list of roles assigned to that user, and if it finds a match, it returns "true," allowing them to see special features that regular users cannot access.



Saved: 5/4/2026 8:02:09 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

≡ Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

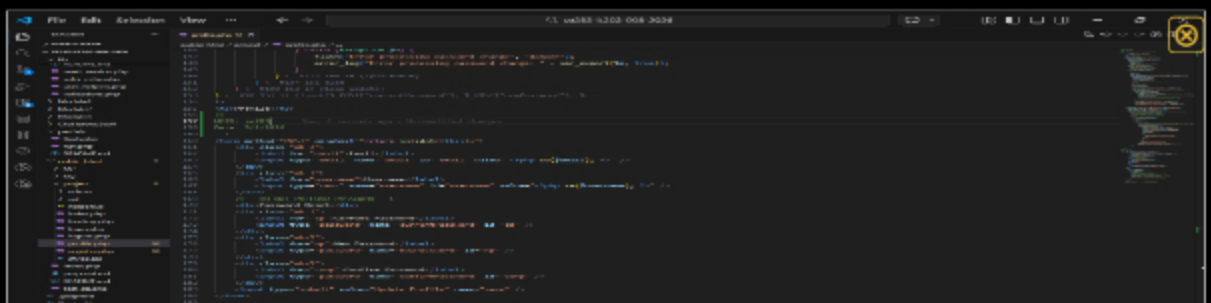
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

🖼️ Part 1:

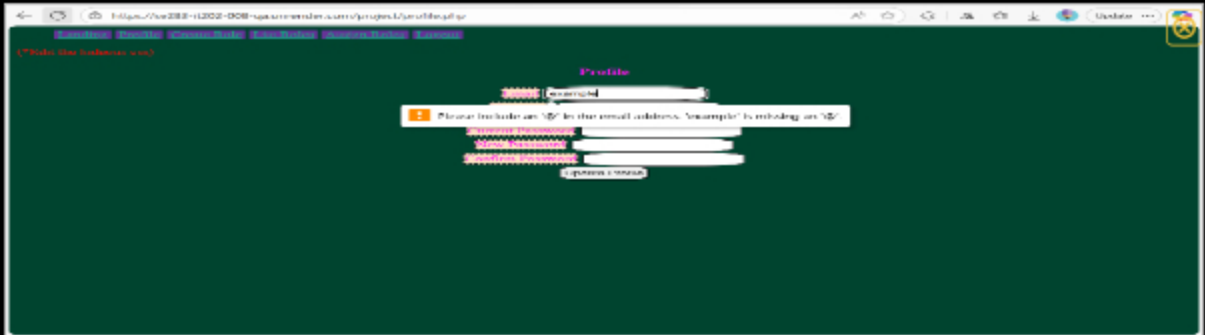
Progress: 100%

Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)



Show the code related to the profile page's form (HTML validation)



Show examples of each validation message [email format]



Saved: 5/4/2026 8:09:27 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

For the profile page, I used standard HTML attributes like `type="email"` and `type="password"` within the input tags. This satisfies the requirement by ensuring the browser handles the basic formatting checks, such as making sure the email field contains an "@" symbol. By using these attributes, the user gets immediate feedback if they try to save an obviously incorrect email address before the form is even sent to the server.



Saved: 5/4/2026 8:09:27 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

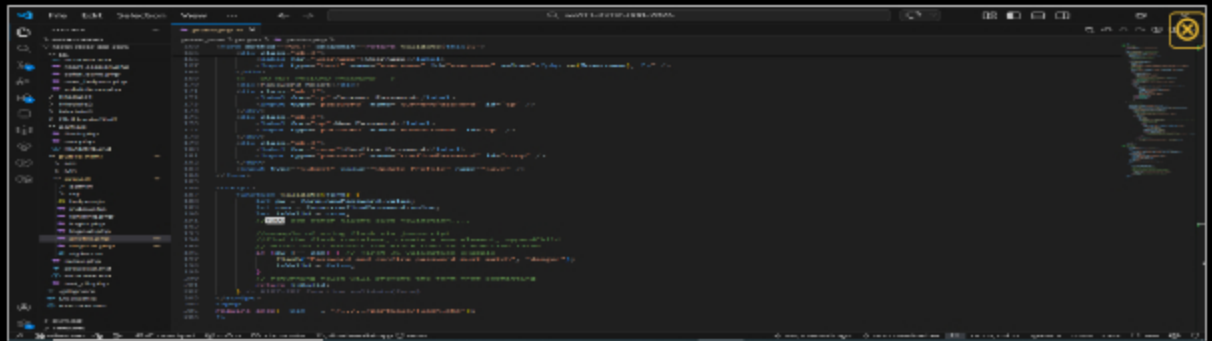
Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's `validate()` function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form

tag



```
function validate() {
    $new_password = $_POST['new_password'];
    $confirm_password = $_POST['confirm_password'];

    if ($new_password === $confirm_password) {
        return true;
    } else {
        return false;
    }
}
```

Show the code related to the profile page's validate() function



Show examples of each validation message [password doesn't match confirm]

 Saved: 5/4/2026 7:53:21 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I implemented a JavaScript validate(form) function that runs as soon as the "Update Profile" button is clicked. This solves the requirement by performing a quick check on the user's computer to ensure that the "New Password" and "Confirm Password" fields are identical. If they don't match, the script stops the update and shows a message, which prevents the user from accidentally locking themselves out with a mistyped password.

 Saved: 5/4/2026 7:53:21 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

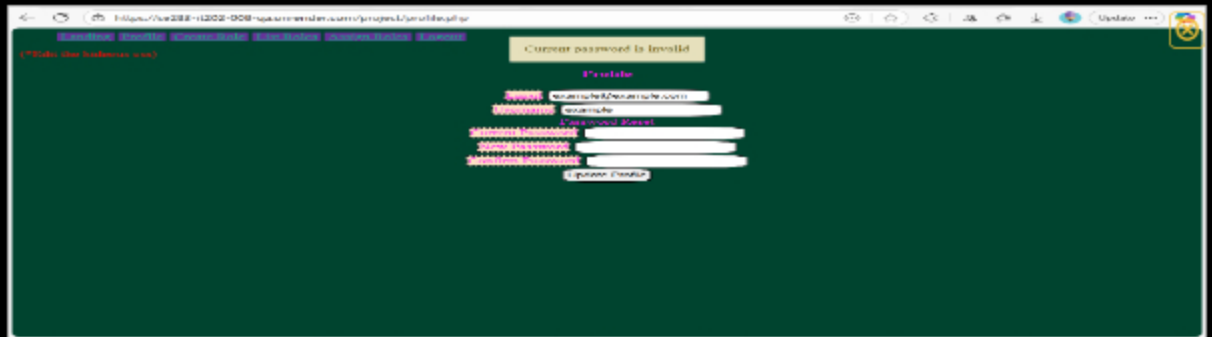
Progress: 100%

Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions

A screenshot of a code editor showing PHP validation code for a profile page. The code is written in PHP and includes various validation functions and error messages. The code is organized into sections for different validation rules, such as username, email, password, and confirm password. The code is color-coded and includes comments.

Show the code related to the profile page's PHP validation



Show examples of each validation message [invalid password]



Show examples of each validation message [updated username]



Show examples of each validation message [username format]

 Saved: 5/4/2026 8:01:12 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I used PHP functions like `is_valid_email()` and `is_valid_username()` to check the data on the server. This solves the requirement by acting as a strict gatekeeper that verifies the new data is safe and follows specific rules (like usernames only having certain characters) before updating the database. It also handles the most critical security step: checking the "Current Password" against the database to make sure only the account owner can make changes.

 Saved: 5/4/2026 8:01:12 PM

⇒ Task #2 (1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

<https://github.com/se283/se283->



URL

<https://github.com/se283/se283->



<https://github.com/se283/se283-> milestone1/public_html/project/profile.php

URL #2

[https://se283-it202-008-](https://se283-it202-008-prod.onrender.com/project/profile.php)



URL

[https://se283-it202-008-prod.](https://se283-it202-008-prod.onrender.com/project/profile.php)



[prod.onrender.com/project/profile.php](https://se283-it202-008-prod.onrender.com/project/profile.php)

URL #3

<https://github.com/se283/se283-it202-008-202613>



URL

<https://github.com/se283/se283-it202-008-202613>



Saved: 5/4/2026 1:20:34 PM

Section #6: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - Github Details

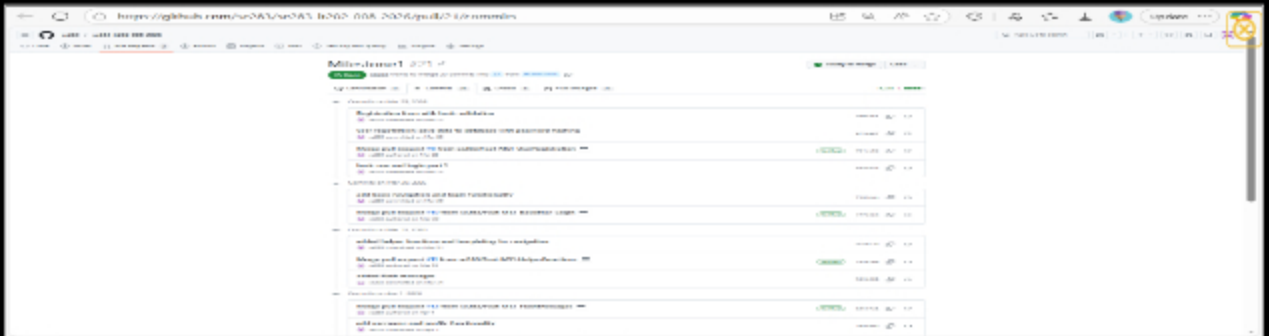
Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reaso



Saved: 5/3/2026 10:37:07 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`) (Milestone1 into qa)

URL #1

<https://github.com/se283/se283-it202-008-202613>



URL

<https://github.com/se283/se283-it202-008-202613>



Saved: 5/3/2026 10:37:07 PM

Task #2 (0.33 pts.) - Waka Time - Activity

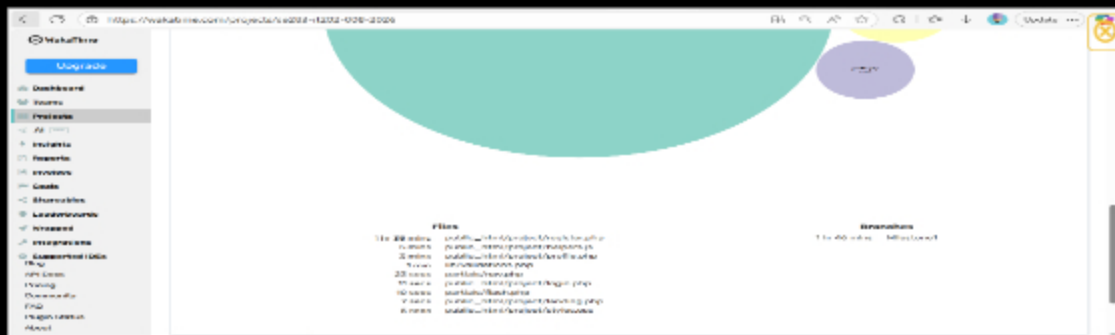
Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



Capture the overall time at the top that includes the repository name



Capture the individual time at the bottom that includes the file time



Saved: 5/4/2026 12:52:18 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how validation works across the different layers of a web application to keep data clean and safe. I learned that HTML validation involves using simple rules in the browser to make sure users don't leave fields empty or enter the wrong email format. I also learned how to use JavaScript validation to perform quick checks, like matching two passwords, before the form is even sent to the server. Finally, I learned that PHP validation checks everything one last time and handles sensitive tasks like hashing passwords and checking for duplicate accounts in the database to make sure only correct information is saved.



Saved: 5/3/2026 10:51:46 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was taking screenshots of my code to show I finished the validation rules. These rules were mostly about how the site checks information when someone creates a new account or updates their profile. It was simple to capture these images to prove that the forms work correctly and keep the user's data safe.



Saved: 5/3/2026 11:10:03 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was determining which file best fit each specific question on the assignment since many files do similar things. For example, because both the login and registration pages use forms and talk to the database, it was tricky to explain how each one separately met the different rules for security and user setup.



Saved: 5/3/2026 10:49:34 PM